

AN INTERNSHIP REPORT

ON

DATA ANALYTICS AND MACHINE LEARNING /

DATA ANALYTICS USING POWER BI TOOL /

MACHINE LEARNING BASED FUEL

CONSUMPTION PREDICITON

Submitted by

Tank Nidhi Govindbhai (210280107023)

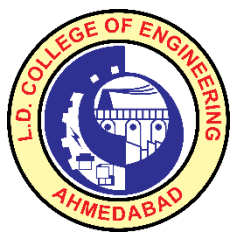
Student of

Bachelors of Engineering

in

Computer Engineering Department

SEM 07

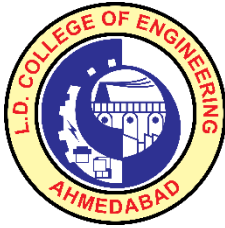


DECLARATION

I do hereby solemnly declare that the work presented in this Internship Report has been carried out by me and has not been previously submitted to any other University, College, and Organization for any academic Qualification and Certificate.

I hereby warrant that the work I have presented does not breach any existing copyright acts.

Student Name Student Sign
Tank Nidhi Govindbhai



CONFIRMATION LETTER



INFOLABZ IT SERVICES PVT. LTD.
WEB DEVELOPMENT | APP DEVELOPMENT | DATA SCIENCE | IOT

Date: 20-06-2025

Subject: Regarding Summer Internship at INFOLABZ IT SERVICES PVT. LTD.
Internship Domain: Data Analytics and Machine Learning

Dear Tank Nidhi Govindbhai,

In reference to your application we would like to congratulate you on being selected for internship with InfoLabz IT Services Pvt. Ltd. based at Ahmedabad. Your internship is scheduled to start from 02 July 2025 for a period of 15 days.

During this internship, the concentration will be on helping you understand logical concepts with their practicality and implications to help you connect your classroom knowledge to industry standards. Your internship will include orientation and focus primarily on learning and developing new skills and gaining a deeper understanding of concepts through hands-on application of the knowledge.

We will be happy to guide you to learn new skills which are extremely helpful in professional standard.

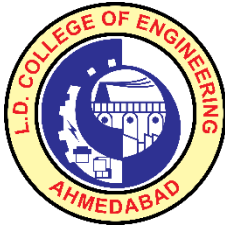


Ms. Twinkle Shah
Internship Coordinator
InfoLabz, Ahmedabad

+91 8866662662
+91 8141236662

info@infolabz.in
www.infolabz.in

405 Vraj Avenue, Above SAM'S Pizza
Nr. Commerce Six Rd, Navrangpura,
Ahmedabad, Gujarat 380009



CERTIFICATE OF COMPLETION



INFOLABZ IT SERVICES PVT. LTD.
WEB DEVELOPMENT | APP DEVELOPMENT | DATA SCIENCE | IOT

COMPLETION CERTIFICATE OF SUMMER INTERNSHIP

Date: 17-07-2025

Enrollment: 210280107023

Semester: 7th, Computer Engineering

L.d College Of Engineering

To whom it may concern,

We are delighted to provide this certificate for the successful completion of the requirements and work performed during the two-week free internship (02 July 2025 to 16 July 2025) by Tank Nidhi Govindbhai.

In this internship tenure, we have covered the fundamentals of Data Analytics and Machine Learning. In the data analytics part, we have worked on API data and covered the basics of data-analysis using pandas and data-visualization using matplotlib and Power BI. In machine learning, we have implemented elementary regression models.

We wish Tank Nidhi Govindbhai all the best for future endeavors.



Ms. Twinkle Shah
Internship Coordinator
InfoLabz, Ahmedabad



+91 8866662662
+91 8141236662



info@infolabz.in
www.infolabz.in



405 Vraj Avenue, Above SAM'S Pizza
Nr, Commerce Six Rd, Navrangpura,
Ahmedabad, Gujarat 380009

COMPANY PROFILE

Established in 2016, incorporation with our parent IT company, INFOLABZ IT SERVICES PVT. LTD. has managed to make it's own position in IT Sector. We are involved in Web Development, App Development, Progressive Web Application Development, IOT solutions, Graphics & Designing, Digital Marketing, Domain & Hosting services, SMS services etc.

In the span of seven years we have managed to deliver all projects on time with utmost accuracy to our clients across the globe. We have dedicated teams of experienced and hard working developers. Our developers who are always willing to take new challenges and looking forward to learn new things, are heart of this company.

Our objective is to sustain with exponential growth in IT industry. Our mission is to deliver the best with top notch quality every quarter and vision is to develop a product with one of its kind concept which could be used by millions of people.

TABLE OF CONTENT

WEEK / DAY NO	CONTENT	PAGE NO
	Declaration	II
	Confirmation Letter	III
	Completion Certificate	IV
	Company Profile	V
WEEK 1/ Day 01	02 JULY 2025	9
	- Types of Data Sources	
	- Dictionary Data Structure	
	- Fetching data from live API , ISRO API , CRYPTO API	
WEEK 1/ Day 02	03 JULY 2024	11
	- Printing data from API using loop	
	- Search Operation on API	
	- Pandas Data Frame	
	- API data to CSV	
WEEK 1/ Day 03	04 JULY 2024	13
	- Operations on Pandas DataFrame	
	- Reading Data from CSV file	
	- Multiple CSV file handling (RESULT ANALYSIS)	
WEEK 1/ Day 04	07 JULY 2024	15

	- Covid Data Analytics	
	- Matplotlib : Bar Graph, Pie Chart, Line Graph	
	- API data to Graph	
WEEK 1/ Day 05	08 JULY 2024	17
	- Machine Learning : Linear Regression Mathematics + Coding	
WEEK 1/ Day 06	09 JULY 2024	19
	- Machine Learning : Multiple Linear Regression Mathematics + coding	
WEEK 2/ Day 07	10 JULY 2024	21
	- PowerBI setup	
	- Basics of Power BI	
WEEK 2/ Day 08	11 JULY 2024	23
	- Graphs and Charts in PowerBI	
WEEK 2/ Day 09	14 JULY 2024	25
	- Data Handling : Advance Power BI	
WEEK 2/ Day 10	15 JULY 2024	26
	- Encoding	
	- Train test split	
	- Random Forest project : Master Degree opt/not opt Prediction	

WEEK 2/ Day 11	16 JULY 2024	30
	- Random Forest Project : Fuel consumption	
	- Conclusion and Project Repository	

WEEK 1/ Day 01**02 JULY 2025** **Objective:**

- Types of Data Sources:
- Dictionary Data Structure
- Fetching data from live API , ISRO API , CRYPTO API

 Types of Data Sources:

S.No	Data Source	Recommendation	Brief Explanation
1	Variable Data	✗ Not Recommended	Data hardcoded as variables in code (e.g., x = [1,2,3]). Not scalable or reusable for real projects.
2	Text Data (.txt)	✗ Not Recommended	Unstructured and lacks tabular format. Hard to analyze or visualize directly. Used only for simple data or logs.
3	CSV / Excel Data	✓ Recommended	Structured, easy to read, widely supported in Python (pandas). Ideal for tabular data (e.g., sales, customer records).
4	Folder Data	✓ Recommended	Useful for image/audio datasets where each file represents a sample (e.g., folders of cat/dog images for classification).
5	LiveData / API	★ Recommended Most	Real-time, dynamic data pulled from web APIs (e.g., stock prices, weather). Great for real-world projects and automation.

 Dictionary Data Structure:

- A dictionary in Python is an **unordered collection of key-value pairs**, used to store data values like a map.
- **Key Features:**
 - Keys are **unique**
 - To access value Key is required
 - Defined using **curly braces {}**

```
In [1]: mydata = { "Ahmedabad": [{"date": "28 June 2025", "cases": 15},
                    {"date": "29 June 2025", "cases": 45},
                    {"date": "30 June 2025", "cases": 35}
                  ],
                "Surat": [300, 350, 1],
                "Rajkot": 400
            }
# print all main keys
print(mydata.keys())

dict_keys(['Ahmedabad', 'Surat', 'Rajkot'])
```

```
In [2]: # print count of main keys

print(len(mydata.keys()))

3
```

```
In [3]: # print 400
print(mydata["Rajkot"])

400
```

Fetching Data from Live APIs:

- **API (Application Programming Interface)** allows two applications to communicate.
- In data science/ML, **APIs are used to fetch real-time data** from the internet, such as weather, financial, or scientific data.
- **How It Works:**
 1. You send a **request** (usually HTTP) to the API endpoint (URL).
 2. The API sends back a **response**, usually in **JSON format**.
 3. You parse this response in Python using libraries like requests.

API / JSON : LIVE / SERVER DATA <=====> OFFLINE DATA : DICTIONARY

ISRO : <https://isro.vercel.app/api/spacecrafts>

```
In [6]: import requests

url = requests.get(" https://isro.vercel.app/api/spacecrafts")
mydata = url.json()

# print(mydata)
print(type(mydata))

<class 'dict'>
```

```
In [7]: # print main keys
print("Main keys in this isro API: ", mydata.keys())

# print number of main keys
print("Number of main keys: ", len(mydata.keys()))

Main keys in this isro API: dict_keys(['spacecrafts'])
Number of main keys: 1
```

```
In [8]: # print Aryabhat

print(mydata["spacecrafts"][0]["name"])

Aryabhata
```

CRYPTO : <https://api.coinlore.net/api/tickers/>

```
In [9]: url = requests.get("https://api.coinlore.net/api/tickers/")
mydata = url.json()

print(mydata.keys())
print(type(mydata))

dict_keys(['data', 'info'])
<class 'dict'>
```

```
In [10]: # print Bitcoin real time price

print("Real time", mydata["data"][0]["name"], "Price:" ,mydata["data"][0]["price_usd"])

Real time Bitcoin Price: 117543.77
```

WEEK 1/ Day 02**03 JULY 2024** **Objective:**

- Printing data from API using loop
- Search Operation on API
- Pandas Data Frame
- API data to CSV

 Printing Data from API Using Loop – In Brief

- When fetching data from an API, the response is often in **JSON format**, which contains **lists or dictionaries**. We use loops to iterate through this structured data and print specific values.

```
In [1]: #api operations
import requests

url = requests.get("https://api.coinlore.net/api/tickers/")
mydata = url.json()

# print(mydata)
print(type(mydata))

# how many main keys are there ?
print(len(mydata.keys()))

#print all main keys
print(mydata.keys())

# allow user to insert key name, print found not found
userkey = input("Enter key name ")
for i in mydata:
    if userkey == i:
        print("key found")
        break
else:
    print("no such key in this api")

# how many crypto coins are there in this API ?
print("total number of coins: ", len(mydata["data"]))

# print name, sr no and price of all coins using for loop
for i in range(0,len(mydata["data"])):
    print( i+1,"Coin Name: ", mydata["data"][i]["nameid"],"Price: ",mydata["data"][i]["price_usd"])
```

 Search Operation on API – In Brief

- **Search operations** allow you to query specific data from an API using keywords, IDs, or parameters. Most APIs provide search endpoints where you can pass a search term to retrieve filtered results.

```
In [2]: # Allow User to insert Coin Name, Print Real time price of that coin only(Search)

coinName = input("Enter coin name: ")
for i in range(len(mydata["data"])):
    if coinName == mydata["data"][i]["name"]:
        print("Current Price of ", coinName, " is ", mydata["data"][i]["price_usd"])
        break
    else:
        print("coin not found")

Enter coin name: Bitcoin
Current Price of Bitcoin is 117932.06
```

Pandas DataFrame

- A **DataFrame** is the core data structure in the **pandas** library. It is a **2D labeled table** (like an Excel sheet) that holds **rows and columns**, making it ideal for data analysis and manipulation.

```
In [3]: import pandas as pd

mydata = [15,25,35]
print(type(mydata))
pddata = pd.DataFrame(mydata)
print(pddata)
print(type(pddata))

newdata = (15,25,35)
print(type(newdata))
newpddata = pd.DataFrame(newdata)
print(newpddata)
print(type(newpddata))

impdata = {"Rohit": [75,12,124], "Virat": [87,45,92]}
print(type(impdata))
impdata = pd.DataFrame(impdata)
print(impdata)
print(type(impdata))

impdata.to_csv("testing.csv")
```

Saving API Data to CSV – In Brief

- After fetching data from an **API** (usually in **JSON format**), we often convert it into a **CSV file** for analysis or record-keeping. This is done using the pandas library.

```
In [4]: # API to CSV

url = requests.get("https://api.coinlore.net/api/tickers/")
mydata = url.json()
print(type(mydata))
coindata = mydata["data"]
#print(coindata)
print(type(coindata))

pdcoindata = pd.DataFrame(coindata)
print(pdcoindata)

pdcoindata.to_csv("Livecoinprice.csv")
```

WEEK 1/ Day 03

04 JULY 2024

**Objective:**

- Operations on Pandas DataFrame
- Reading Data from CSV file
- Multiple CSV file handling (RESULT ANALYSIS)

**Operations on Pandas DataFrame**

- Pandas provides powerful tools to perform a variety of operations on DataFrames to **analyze, clean, and manipulate** data efficiently.

```
In [1]: import pandas as pd
import numpy as np

# Creating a DataFrame with performance scores of Rohit and Virat
impdata = {"Rohit": [75,12,124], "Virat": [87,45,92]}
impdata = pd.DataFrame(impdata, index=[1,2,3]) #customized index
print("Player Scores DataFrame:")
print(impdata)
print()

# Creating a DataFrame with player roles and scores in a particular game
game = pd.DataFrame([["Rohit","Batsman",50],["Kohli","Batsman",75]],
                    columns=["Name", "Type","Scores"], index =[1,2])
#customized index starts with 1
print("Game Performance DataFrame:")
print(game)
print()

# Creating a NumPy array of salaries over years for two employees
empdata = np.array([[8000,10000,18000],
                    [15000,18000,25000]])

# Converting NumPy array into a DataFrame with proper column and row labels
pdempdata = pd.DataFrame(empdata,
                        columns=[2022,2023,2024],
                        index=["Ramesh", "Mahesh"])
print("Employee Salary DataFrame (2022-2024):")
print(pdempdata)
print()

# Adding a new column for 2025 with a 30% increment over 2024 salary
# pdempdata[2025]= (pdempdata[2024]*0.3) + pdempdata[2024]
pdempdata[2025] = pdempdata[2024] * 1.30
print("Employee Salary DataFrame (including 2025 with 30% increment):")
print(pdempdata)
```

★ .loc[] vs .iloc[] in Pandas

.loc[] – Label-based indexing

- Used to access rows and columns by label names.
- Includes both start and end labels when slicing.
- **Syntax: df.loc[row_label, column_label]**

.iloc[] – Position-based indexing

- Used to access rows and columns by integer positions (like Python lists).
- End index is exclusive when slicing.
- **Syntax: df.iloc[row_position, column_position]**

📁 Reading Data from CSV File – In Brief

- **CSV (Comma Separated Values)** is a common file format for storing tabular data. In Python, the pandas library provides a simple way to read CSV files into a **DataFrame** for analysis.

```
In [2]: # Reading data from a CSV file named "Livecoinprice.csv" into a DataFrame
filedata = pd.read_csv("Livecoinprice.csv")

# Printing the row at index 2 from the DataFrame using .loc (Label-based indexing)
# Note: .loc[[2]] returns the row where the index label is 2 (not necessarily the third row unless index is default)
print(filedata.loc[[2]])
```

```
In [3]: filedata = pd.read_csv("mydata.xlsx - Sheet1.csv")
print(filedata)
print(type(filedata))
print()

# Print Score of all matches of Kohli
print(filedata["KOHLI"])
print()

# Print 37
# Specifica: Varname[Column][Row]
print(filedata["DHONI"][3])
print()

# Print Third Match Data
print(filedata.loc[[2]])
print(filedata.iloc[[2]])
# As the default index starts with 0, 3rd Match is in index 2
# here label name and position both are 2 so both will print the same result
```

📁 Handling Multiple CSV Files (Result Analysis) – In Brief

- In many real-world scenarios, data is split across **multiple CSV files** (e.g., results by subject, class, or semester). Using Python and pandas, you can **read, combine, and analyze** these files efficiently.

```
In [4]: df1 = pd.read_csv("RESULT1 - Sheet1.csv")
print(df1)

df2 = pd.read_csv("RESULT2 - Sheet1.csv")
print(df2)

alldata = pd.concat([df1, df2])
alldata
```

```
In [5]: alldata.head(3)
```

```
Out[5]:
```

	SRNO	BRANCH	NAME	TOTAL	PERCENTAGE	PASSFAIL
0	1	CE	RAMESH	210	90	1
1	2	CE	SURESH	150	80	1
2	3	IT	MAHESH	225	75	1

```
In [6]: #sort data based on Total
print(alldata.sort_values(['TOTAL'], ascending = False).head(3))
```

	SRNO	BRANCH	NAME	TOTAL	PERCENTAGE	PASSFAIL
2	3	IT	KATHAN	285	95	1
1	2	CE	JATAN	270	90	1
2	3	IT	MAHESH	225	75	1

```
In [7]: print(alldata.sort_values(['TOTAL'], ascending = False).tail(3))
```

	SRNO	BRANCH	NAME	TOTAL	PERCENTAGE	PASSFAIL
1	2	CE	SURESH	150	80	1
0	1	EC	RATAN	150	50	1
4	5	CE	JAYESH	90	30	0

```
In [8]: df1.dtypes
```

```
Out[8]: SRNO      int64
BRANCH    object
NAME      object
TOTAL     int64
PERCENTAGE int64
PASSFAIL  int64
dtype: object
```

WEEK 1/ Day 04**07 JULY 2024** **Objective:**

- Covid Data Analytics
- Matplotlib : Bar Graph, Pie Chart, Line Graph
- API data to Graph

 COVID Data Analytics

- It involves analyzing pandemic-related data to uncover patterns, trends, and insights about the spread and impact of the virus.
 - Daily confirmed, discharged, and death cases
 - State-wise case distribution

```
In [1]: import requests
import pandas as pd

url = requests.get("https://api.rootnet.in/covid19-in/stats/latest")
mydata = url.json()

#print(mydata)
# print all main keys
print(mydata.keys())

# print total number of main keys
print("total number of keys: ", len(mydata.keys()))

# print Andaman and Nicobar islands
print(mydata['data']['regional'][0]['loc'])

# print total number of states in this API
print(len(mydata['data']['regional']))

# allow user to insert state name, Print state found not found
userinput = input("Enter state name: ")
for i in range(0, len(mydata['data']['regional'])):
    if userinput == mydata['data']['regional'][i]['loc']:
        print("State found")
        print("Total Confirmed: ", mydata['data']['regional'][i]['totalConfirmed'])
        print("Total Discharged: ", mydata['data']['regional'][i]['discharged'])
        print("Total Deaths: ", mydata['data']['regional'][i]['deaths'])
        break
    else:
        print("State Not found")

dict_keys(['success', 'data', 'lastRefreshed', 'lastOriginUpdate'])
total number of keys: 4
Andaman and Nicobar Islands
36
Enter state name: Gujarat
State found
Total Confirmed: 1224657
Total Discharged: 1213502
Total Deaths: 10944

In [2]: coviddata = pd.DataFrame(mydata['data']['regional'])
coviddata.head()

In [3]: coviddata.to_csv("covid.csv")

In [4]: # Conditional data fetching:
covidpd = pd.read_csv("covid.csv")

# States having cases between 100000 to 150000
covidpd[(covidpd["totalConfirmed"]>=100000)& (covidpd["totalConfirmed"]<= 150000)]
```



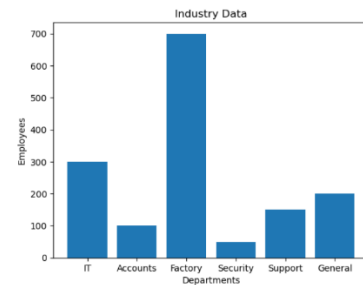
Matplotlib – Bar Graph, Pie Chart, Line Graph (In Brief)

- It is a Python library used for creating static, animated, and interactive plots. It is widely used for data visualization in data science and ML projects. Python code and it's respective Outputs:

In [6]: `import matplotlib.pyplot as plt`

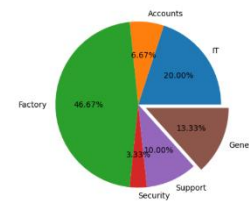
```
In [7]: # Bar Graph
departments = ["IT", "Accounts", "Factory", "Security", "Support", "General"]
employees = [300, 100, 700, 50, 150, 200]

plt.bar(departments, employees)
plt.title("Industry Data")
plt.xlabel("Departments")
plt.ylabel('Employees')
plt.show()
```



In [8]: `# pie chart`

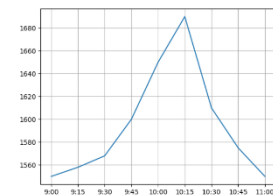
```
plt.pie(employees, labels =departments, autopct="%.2f%%", explode=[0,0,0,0,0,0.1])
plt.show()
```



In [9]: `# Line graph`

```
time = ['9:00', '9:15', '9:30', '9:45', '10:00', '10:15', '10:30', '10:45', '11:00']
reliance = [1550, 1558, 1568, 1600, 1650, 1690, 1610, 1575, 1550]

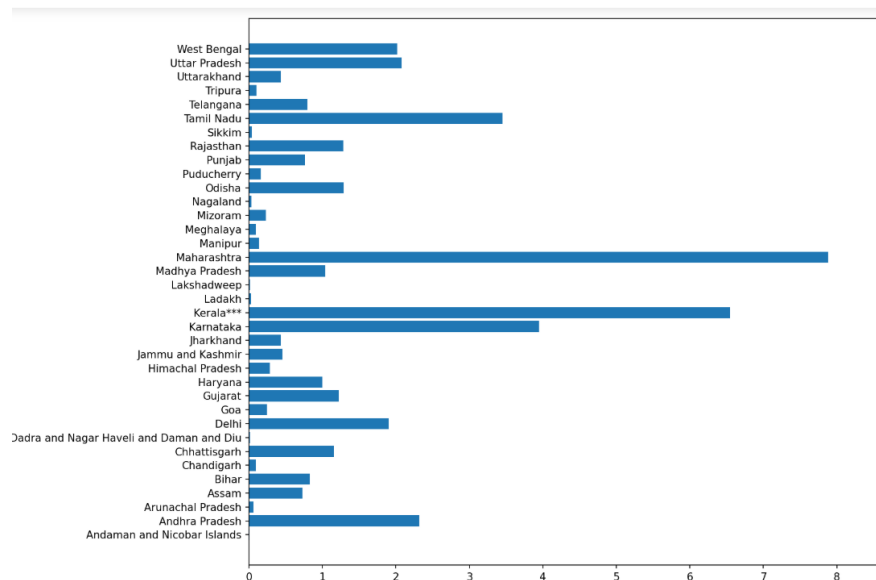
plt.plot(time, reliance)
plt.grid(True)
plt.show()
```



API Data to Graph: [CovidDataset]

In [10]: `# horizontal Bar Chart`

```
covidpd = pd.read_csv("covid.csv")
plt.barh(covidpd['loc'], covidpd['totalConfirmed'])
plt.title("Covid Cases Analysis")
# plt.xlabel("Covid Cases")
# plt.ylabel("States")
plt.show()
```



WEEK 1/ Day 05**08 JULY 2024****Objective:**

- Machine Learning : Linear Regression
(Mathematics + Coding)

**What is Linear Regression?**

Linear Regression is the **simplest form** of supervised learning used for predicting continuous numerical values.

Types of Linear Regression

- **Simple Linear Regression:** 1 independent variable
- **Multiple Linear Regression:** 2 or more independent variables

We aim to find the **best-fit straight line** through the data points.

**Mathematical Model**

For **Simple Linear Regression**, the model predicts output $\hat{y}$ using:

$$[\hat{y} = \beta_0 + \beta_1 x]$$

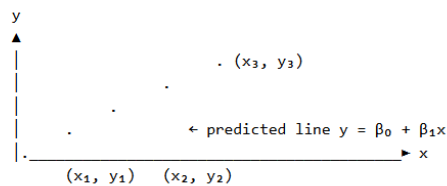
Where:

- $\hat{y}$: Predicted value (output)
- x : Input feature
- β_0 : Intercept (bias)
- β_1 : Slope (weight)

**Graphical Interpretation**

- The X-axis represents the input feature x
- The Y-axis represents the predicted output $\hat{y}$
- The best-fit line: $y = \beta_0 + \beta_1 x$
- Each point contributes a vertical error (called **residual**) to the line

Example visualization:



Data -> Visualization Data Pattern/ Data Nature Helps ML Algorithm

Data -> Model -> Train -> Predict

EQUATION OF SLOPE

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

	x	y	(x - $\bar{x}$)	(y - $\bar{y}$)	(x - $\bar{x}$) ²
	1	2			
	2	4			
	3	5			
	4	4			
	5	5			
Σ					
	$\bar{x} =$	$\bar{y} =$			

x	y	(x - $\bar{x}$)	(y - $\bar{y}$)	(x - $\bar{x}$) ²	(x - $\bar{x}$) x (y - $\bar{y}$)
1	2	-2	-2	4	4
2	4	-1	0	1	0
3	5	0	1	0	0
4	4	1	0	1	0
5	5	2	1	4	2
Σ	15	20		10	6
$\bar{x}$	$\bar{y}$	3	4		

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

$$= 6 / 10$$

$$B_1 = 0.6$$

Put all values in slope equation ($\bar{y}$, B_1 , X mean)

$$\bar{y} = B_0 + B_1 X$$

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt

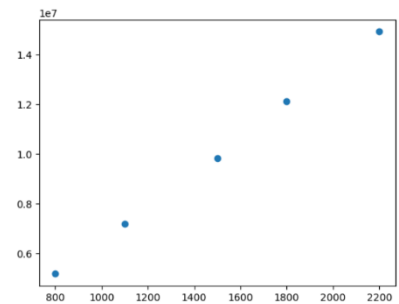
df = pd.read_csv("areaPriceDay5.csv")
df
```

Out[1]:

	area	price
0	800	5180000
1	1100	7190000
2	1500	9820000
3	1800	12100000
4	2200	14900000

```
In [2]: plt.scatter(df['area'], df['price'])
```

Out[2]: <matplotlib.collections.PathCollection at 0x26c5cdddcf0>



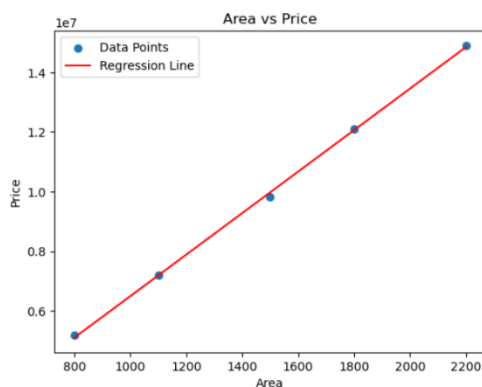
```
In [3]: import matplotlib.pyplot as plt
import numpy as np

# Scatter plot
plt.scatter(df['area'], df['price'], label='Data Points')

# Fit a straight line (linear regression)
m, b = np.polyfit(df['area'], df['price'], 1) # y = mx + b

# Plot the regression line
plt.plot(df['area'], m * df['area'] + b, color='red', label='Regression Line')

# Labels and Legend
plt.xlabel('Area')
plt.ylabel('Price')
plt.title('Area vs Price')
plt.legend()
plt.show()
```



WEEK 1/ Day 06

09 JULY 2024

**Objective:**

- Machine Learning : Multiple Linear Regression
(Mathematics + coding)

**Multiple linear regression in machine learning**

- Multiple linear regression (MLR) is a supervised learning algorithm that aims to predict a continuous **dependent variable** (also known as the **target** or response variable) based on the values of two or more **independent variables** (also known as **features** or predictors).

REGRESSION LINE EQUATION

Multiple Regression Model with k Independent Variables:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \varepsilon$$

Labels: Y-intercept, Population slopes, Random Error

LINEAR REGRESSION WITH 2 INDEPENDENT VAR.

$$a = \bar{Y} - b_1 \bar{X}_1 - b_2 \bar{X}_2$$

$$b_1 = \frac{(\sum x_1^2)(\sum x_1 y) - (\sum x_1 x_2)(\sum x_2 y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1 x_2)^2}$$

$$b_2 = \frac{(\sum x_1^2)(\sum x_2 y) - (\sum x_1 x_2)(\sum x_1 y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1 x_2)^2}$$

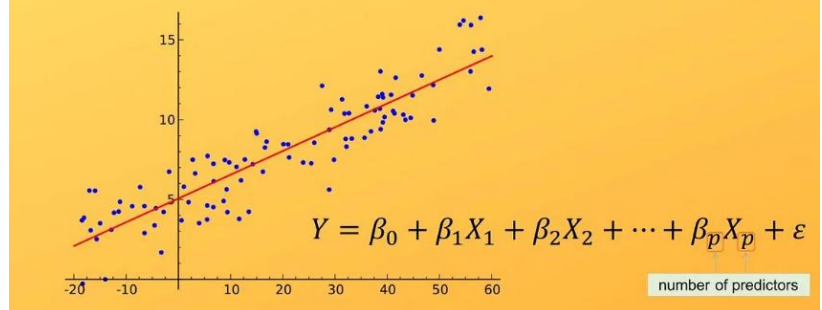
$$\sum x_1^2 = \sum X_1 X_1 - \frac{(\sum X_1)(\sum X_1)}{N}$$

$$\sum x_2^2 = \sum X_2 X_2 - \frac{(\sum X_2)(\sum X_2)}{N}$$

$$\sum x_1 y = \sum X_1 Y - \frac{(\sum X_1)(\sum Y)}{N}$$

$$\sum x_2 y = \sum X_2 Y - \frac{(\sum X_2)(\sum Y)}{N}$$

$$\sum x_1 x_2 = \sum X_1 X_2 - \frac{(\sum X_1)(\sum X_2)}{N}$$

Multiple Linear Regression

1. The general form of the multiple linear regression model
2. Manual formula derivation and calculation steps
3. Step-by-step summation and coefficient calculation
4. Final equation interpretation and graph visualization

INDEPENDENT		DEPENDENT					
X ₁	X ₂	Y	X ₁ X ₁	X ₂ X ₂	X ₁ X ₂	X ₁ Y	X ₂ Y
3	8	-3.7	9	64	24	-11.1	-29.6
4	5	3.5	16	25	20	14	17.5
5	7	2.5	25	49	35	12.5	17.5
6	3	11.5	36	9	18	69	34.5
2	1	5.7	4	1	1	11.4	5.7
Σ	20	24	19.5	90	148	99	45.6

$$b_1 = \frac{(\sum x_2^2)(\sum x_1 y) - (\sum x_1 x_2)(\sum x_2 y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1 x_2)^2} = \frac{32.8 * 17.8 - 3 * (-48)}{10 * 32.8 - 3 * 3} = 2.28$$

$$b_2 = \frac{(\sum x_1^2)(\sum x_2 y) - (\sum x_1 x_2)(\sum x_1 y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1 x_2)^2} = \frac{10 * (-48) - 3 * 17.8}{10 * 32.8 - 3 * 3} = -1.67$$

$$a = \bar{Y} - b_1 \bar{X}_1 - b_2 \bar{X}_2 = \frac{19.5}{5} - \frac{2.28 * 20}{5} - \frac{-1.67 * 24}{5} = 2.79$$

Final Regression equation or Model is:

$$Y = 2.796 + 2.28 x_1 - 1.67 x_2$$

Multiple Linear Regression Jupyter Notebook Code:

Dataset: Boston Housing (predict median home value from multiple features)

```
In [1]: from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error
import numpy as np
import pandas as pd

# Load data
california = fetch_california_housing()
X_full = pd.DataFrame(california.data, columns=california.feature_names)
y_full = pd.Series(california.target, name='MedHouseValue')

# Select 2 features for MLR
X = X_full[['MedInc', 'AveRooms']]
y = y_full
print(X)

# Split into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Fit model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
rmse = root_mean_squared_error(y_test, y_pred)

# Results
print(f"Intercept: {model.intercept_.4f}")
print("Coefficients:", dict(zip(X.columns, model.coef_)))
print(f"RMSE: {rmse:.4f}")
```

```
# Define the input data
input_data = [[7.2574, 8.288136]]

# Convert to DataFrame with the same column names as the training data
input_df = pd.DataFrame(input_data, columns=['MedInc', 'AveRooms'])

# Make the prediction
y_pred = model.predict(input_df)
print("Predicted MedHouseValue:", y_pred[0])
```

```
MedInc AveRooms
0 8.3252 6.984127
1 8.3014 6.238137
2 7.2574 8.288136
3 5.6431 5.817352
4 3.8462 6.281853
...
20635 1.5603 5.045455
20636 2.5568 6.114035
20637 1.7000 5.205543
20638 1.8672 5.329513
20639 2.3886 5.254717

[20640 rows x 2 columns]
Intercept: 0.5973
Coefficients: {'MedInc': 0.4362608864728618, 'AveRooms': -0.04017160872638347}
RMSE: 0.8379
Predicted MedHouseValue: 3.430439780418422
```

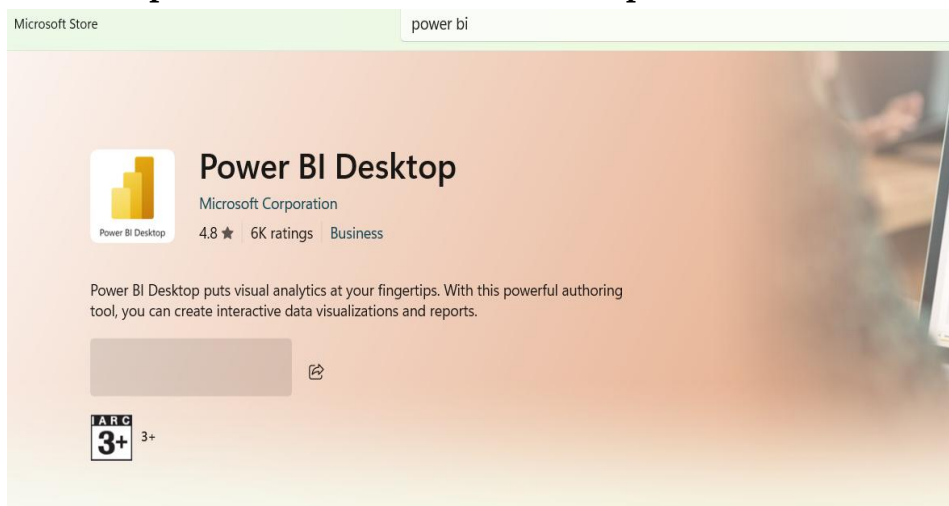
- Output

WEEK 2/ Day 07**10 JULY 2024****Objective:**

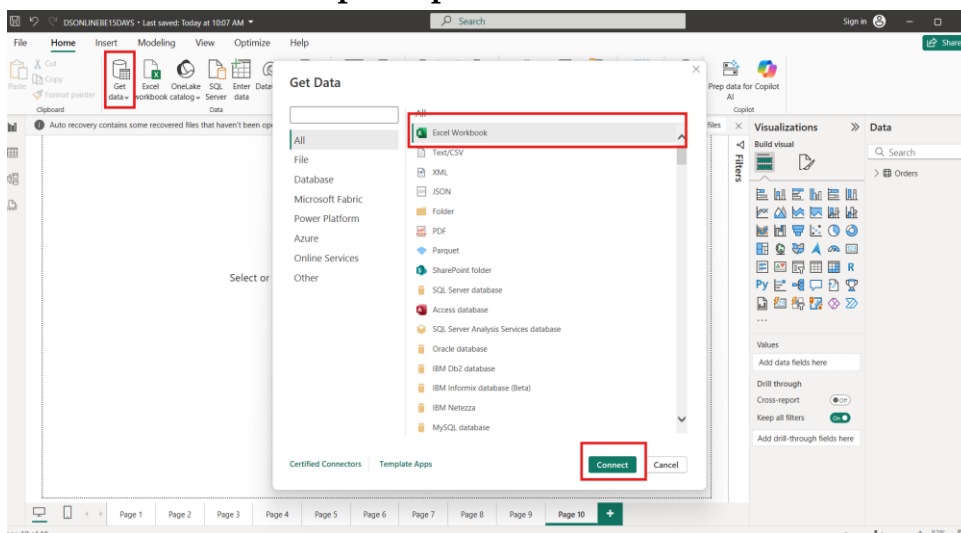
- PowerBI setup
- Basics of Power BI

Power BI Setup:

- **Power BI** is a business analytics tool by Microsoft used to create interactive visualizations and dashboards.
- **Step 1 : Download Power BI Desktop from Microsoft Store.**



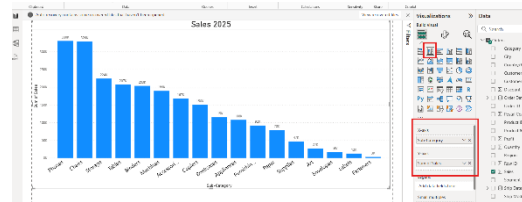
- **Step 2 : Load excel file in power bi.**
Download Sample Superstore file: [Click here](#)



Basics of Power BI:

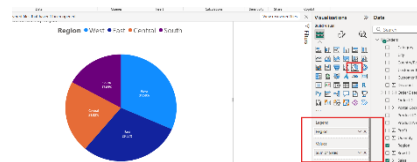
1. Select **Bar Chart** From Visuals

- X Axis - Sub Category
- Y Axis – Sales



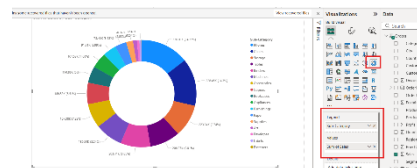
2. Select **Pie Chart** From Visuals

- Legend : Region
- Values: Sales



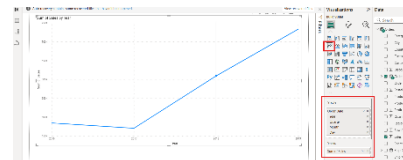
3. Select **Donut Chart** From Visuals

- Legend : Sub Category
- Values: Sales



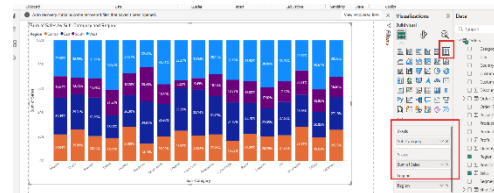
4. Select **Line Chart** From Visuals

- X Axis - Order Date
- Y Axis – Sales



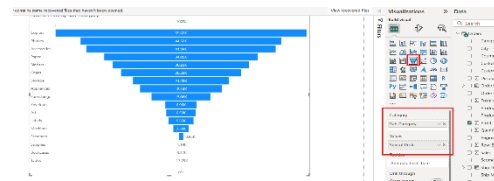
5. Select **Stack Bar Chart** From Visuals

- X Axis - Sub Category
- Y Axis - Sales
- Legend – Region



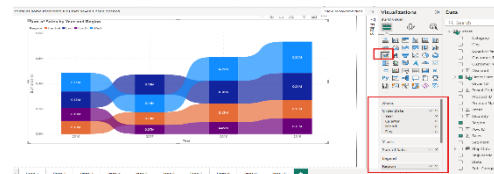
6. Select **Funnel Chart** From Visuals

- Category : Sub-Category
- Values: Profit



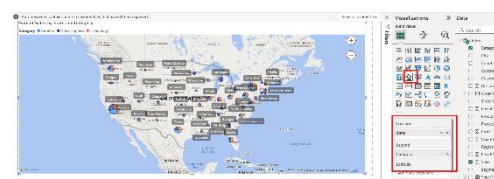
7. Select **Ribbon Chart** From Visuals

- X Axis - Order Date
- Y Axis – Sales



8. Select **Map Chart** From Visuals

- Location: State
- Legend – Category



WEEK 2/ Day 08**11 JULY 2024****Objective:**

- Graphs and Charts in PowerBI

Download India xlsx file: [Click Here](#)

9. Select Map Chart From Visuals

- Location: State
- Legend – Population

**10. Select Table Chart From Visuals**

- Columns: subcategory , sales , profit , discount

Sub-Category	Sum of Sales	Sum of Profit	Sum of Discount
Phones	3,30,007.05	44,515.73	137.40
Chairs	3,28,449.10	26,590.17	105.00
Storage	2,23,843.61	21,278.83	63.20
Tables	2,06,965.53		83.35
Binders	2,03,412.73	30,221.76	567.00
Machines	1,89,238.63	3,364.78	35.20
Accessories	1,67,380.32	41,936.64	60.80
Copiers	1,49,528.03	55,617.82	11.00
Bookcases	1,14,880.00	-3,472.56	48.14
Appliances	1,07,532.16	18,138.01	77.60
Furnishings	91,705.16	13,059.14	132.40
Paper	78,479.21	34,053.57	102.60
Supplies	46,673.54	-1,189.10	14.60
Art	27,118.79	6,527.79	59.60
Envelopes	16,476.40	6,964.18	20.40
Labels	12,486.31	5,546.25	25.00
Fasteners	3,024.28	949.52	17.80
Total	22,97,200.86	2,86,397.02	1,561.09

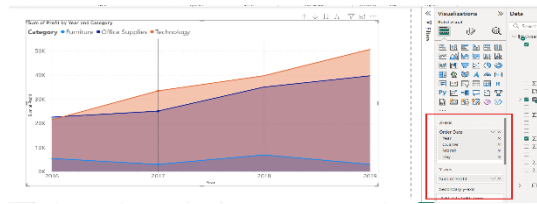
11. Select Matrix Chart From Visuals

- Rows : Sub Category
- Columns : Order Date
- Values: Sales

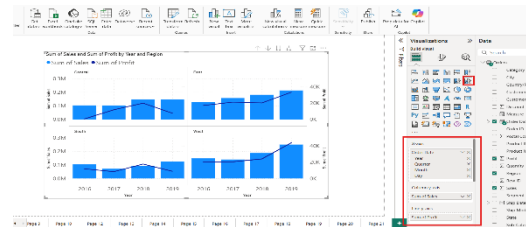
Sub-Category	2016	2017	2018	2019	Total
Accessories	25,014.27	40,523.96	41,895.85	59,946.23	1,67,380.32
Appliances	15,313.63	23,241.29	26,050.32	42,926.93	1,07,532.16
Art	6,057.98	6,236.83	5,960.91	8,863.07	27,118.79
Binders	43,488.27	37,453.10	49,683.33	72,788.04	2,03,412.73
Bookcases	20,036.68	38,543.57	26,275.47	30,024.28	1,14,880.00
Chairs	77,241.58	71,734.53	83,918.64	95,554.35	3,28,449.10
Copiers	10,849.78	26,179.45	49,599.41	62,899.39	1,49,528.03
Envelopes	3,855.75	4,512.19	4,729.89	3,378.57	16,476.40
Fasteners	661.33	545.22	960.13	857.59	3,024.28
Furnishings	13,826.33	21,089.71	27,874.12	28,915.09	91,705.16
Labels	2,841.39	2,956.46	2,827.24	3,861.22	12,486.31
Machines	62,023.37	27,763.70	55,906.89	43,544.68	1,89,238.63
Paper	14,834.96	15,287.64	20,661.89	27,694.72	78,479.21
Phones	77,390.81	68,313.70	78,962.03	1,05,340.52	3,30,007.05
Storage	50,329.04	45,048.25	58,788.70	69,677.62	2,23,843.61
Supplies	14,394.07	1,952.48	14,277.58	16,049.41	46,673.54
Tables	46,088.37	39,150.42	60,833.20	60,893.54	2,06,965.53
Total	4,84,247.50	4,70,532.51	6,09,205.60	7,33,215.26	22,97,200.86

12. Select **Area Chart** From Visuals

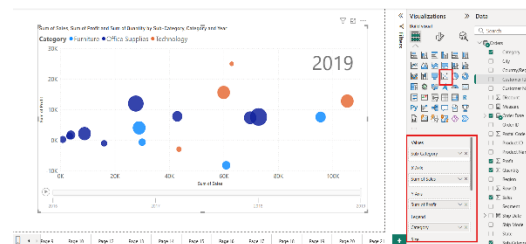
- X Axis : Order Date
- Y Axis : Profit
- Legend: Category

13. Select **Line and Column Chart**

- X Axis: Order Date
- Column Y Axis- Sales
- Line Y Axis: Profit
- Small Multiple: Region

14. Select **Scatter Chart** From Visuals

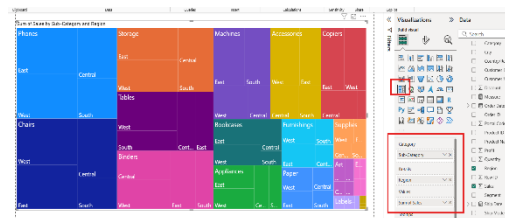
- Values: SubCategory
- X Axis: Sales
- Y Axis: Profit
- Legend: Category
- Size: Quantity
- Play Axis: Order Date (Date Hierarchy)

15. Select **WaterFall Chart** From Visuals

- Category: Order Date
- Breakdown: Region
- Y Axis- Sales

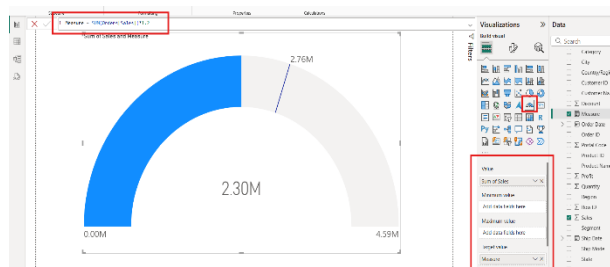
16. Select **Tree Chart** From Visuals

- Category: Subcategory
- Value- Sales
- Details: Region



17. Select Gauge Chart From Visuals

- Value: Sales
- Target Value: Measure = $SUM(Orders[Sales]) * 1.2$

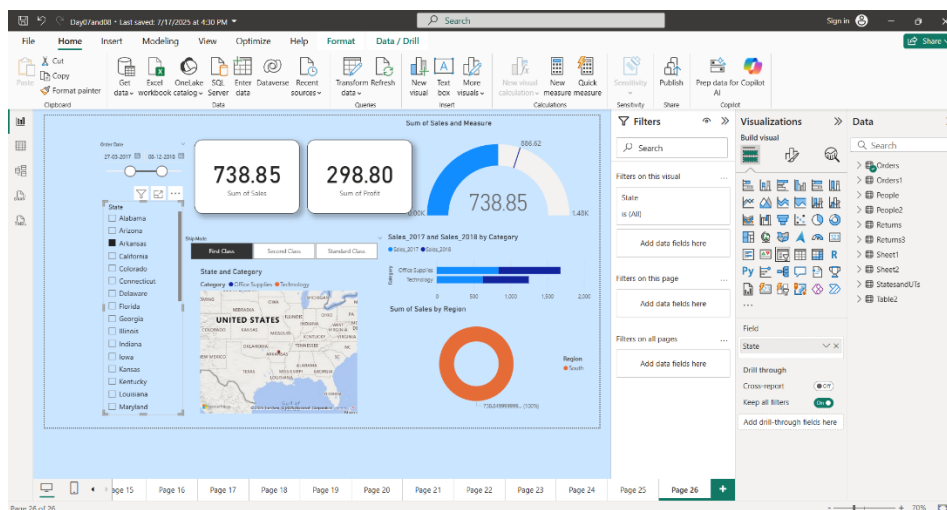


WEEK 2/ Day 09**14 JULY 2024****Objective:**

- Data Handling : Advance Power BI (Dashboard Design)

1. Project Purpose

- The main objective of creating this Power BI dashboard was to analyze sales and profit performance based on several business dimensions such as **time period, state-wise sales, shipping method, product categories, and regional distribution.**
- This report aims to provide meaningful insights to support business decisions.

**2. Dashboard Design Highlights**

Component	Purpose
Date Slicer	Filter data dynamically by order date
State Slicer	Focus on sales by individual states
Ship Mode Buttons	Segment sales/profits by delivery method
KPI Cards (Sales/Profit)	Quick summary of total sales and profit
Gauge Chart	Visualization of current sales against performance target
Bar Chart	Year-wise and category-wise comparison of sales
Map Visualization	Geographic distribution of sales in the United States
Donut Chart	Regional sales contribution in percentage

WEEK 2/ Day 10**15 JULY 2024****Objective:**

- | |
|---|
| - Encoding |
| - Train test split |
| - Random Forest project : Master Degree opt/not opt Prediction |

Masters Decision Support System:

Project Overview

This Streamlit-based project predicts whether a student should pursue a Master's degree based on their GATE score and current salary using a machine learning model. It features a clean user interface with data visualization, prediction form, and dataset preview.

Features in the Application

- Home: Welcome screen and navigation
- About Model: Describes model type and purpose
- Dataset: Shows the contents of dataset.csv
- Prediction Model: Predicts if a user should pursue a Master's
- Graph: Visualizes salary, GATE score, and decision distribution

Machine Learning Model Details

- Model Type: Random Forest Classifier
- Features Used: GATE Score (Categorical), Salary (Numerical)
- Target Column: Should_Do_Masters
- Data Preprocessing: Label Encoding of categorical columns
- Train-Test Split: 80% training, 20% testing

```

1 import streamlit as st
2 from streamlit_option_menu import option_menu
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.preprocessing import LabelEncoder
5 from sklearn.model_selection import train_test_split
6 import pandas as pd
7 import matplotlib.pyplot as plt
8
9 # Load and prepare data
10 model_df = pd.read_csv("dataset.csv")
11
12 le_gate = LabelEncoder()
13 model_df["GATE_Score"] = le_gate.fit_transform(model_df["GATE_Score"])
14
15 le_master = LabelEncoder()
16 model_df["Should_Do_Masters"] = le_master.fit_transform(model_df["Should_Do_Masters"])
17
18 if le_master.transform(['Yes'])[0] != 1:
19     model_df["Should_Do_Masters"] = 1 - model_df["Should_Do_Masters"]
20     le_master = LabelEncoder()
21     le_master.fit(["No", "Yes"])

```

```

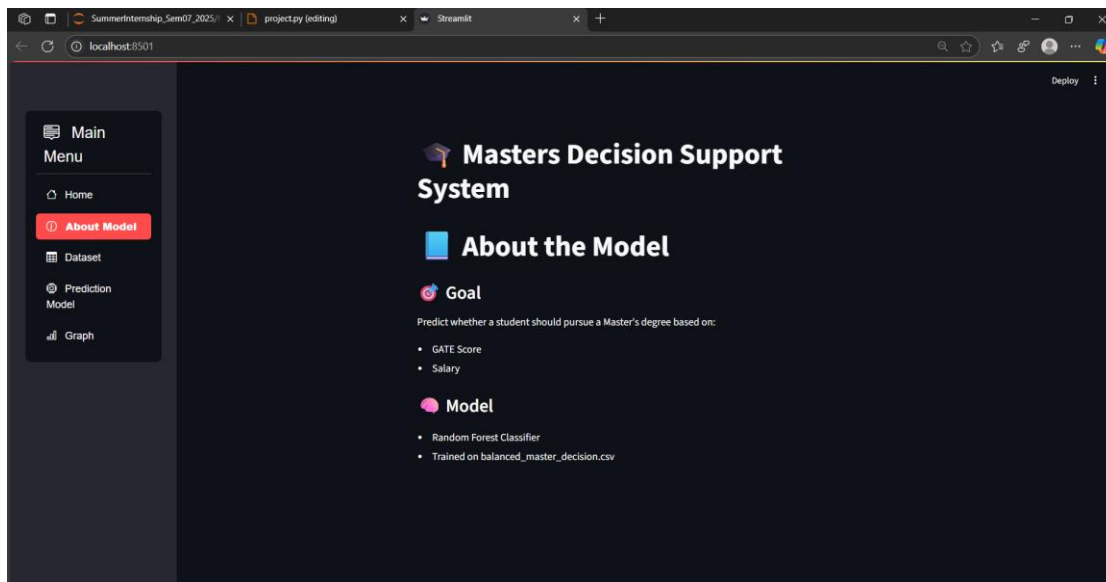
22 gate_label_map = dict(zip(le_gate.transform(le_gate.classes_), le_gate.classes_))
23 master_label_map = {0: "No", 1: "Yes"}
24
25
26 X = model_df[["GATE_Score", "Salary"]]
27 y = model_df["Should_Do_Masters"]
28
29 x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
30 model = RandomForestClassifier(n_estimators=500, random_state=42)
31 model.fit(x_train, y_train)
32
33 # UI
34 st.title("🎓 Masters Decision Support System")
35
36 with st.sidebar:
37     selected = option_menu(
38         menu_title="Main Menu",
39         options=["Home", "About Model", "Dataset", "Prediction Model", "Graph"],
40         icons=["house", "info-circle", "table", "cpu", "bar-chart"],
41         orientation="vertical"
42     )
43
44 # Pages
45 if selected == "Home":
46     st.subheader("👋 Welcome!")
47     st.write("Use the sidebar to explore the application.")
48
49 elif selected == "About Model":
50     st.title("📄 About the Model")
51     st.markdown("""
52     ### 🎯 Goal
53     Predict whether a student should pursue a Master's degree based on:
54     - GATE Score
55     - Salary
56
57     ### 🧠 Model
58     - Random Forest Classifier
59     - Trained on balanced_master_decision.csv
60
61     """)
62
63 elif selected == "Dataset":
64     st.title("📁 Uploaded Dataset (master_decison.csv)")
65     try:
66         dataset_df = pd.read_csv("dataset.csv")
67         st.dataframe(dataset_df)
68     except FileNotFoundError:
69         st.error("data.csv file not found. Please upload or check path.")
70
71
72 elif selected == "Prediction Model":
73     st.title("🔮 Predict: Should You Do a Master's?")
74     gate_input = st.selectbox("Select GATE Score", le_gate.classes_)
75     salary_input = st.number_input("Enter current salary (INR)", value=500000)
76
77     if st.button("Predict"):
78         gate_encoded = le_gate.transform([gate_input])[0]
79         prediction = model.predict([[gate_encoded, salary_input]])
80         result = le_master.inverse_transform(prediction)[0]
81         st.success(f"Prediction: You should {'🎓 do' if result == 'Yes' else '🚫 not do'} a Master's.")
82
83 elif selected == "Graph":
84     st.title("📊 Visualize Data from master_decison.csv")
85     feature = st.selectbox("Choose a feature", ["GATE_Score", "Salary", "Should_Do_Masters"])
86
87     if feature == "GATE_Score":
88         st.subheader("GATE Score Distribution")
89         counts = model_df[feature].value_counts().sort_index()
90         labels = [gate_label_map[i] for i in counts.index]
91         plt.figure()
92         plt.bar(labels, counts.values, color="purple")
93         plt.xlabel("GATE Score")
94         plt.ylabel("Count")
95         st.pyplot(plt.gcf())

```

```

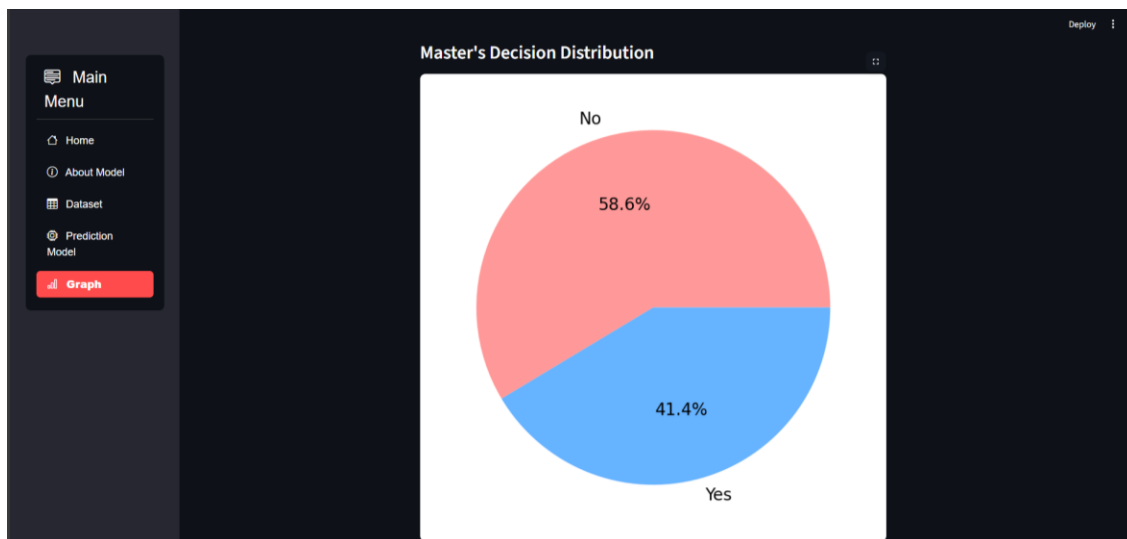
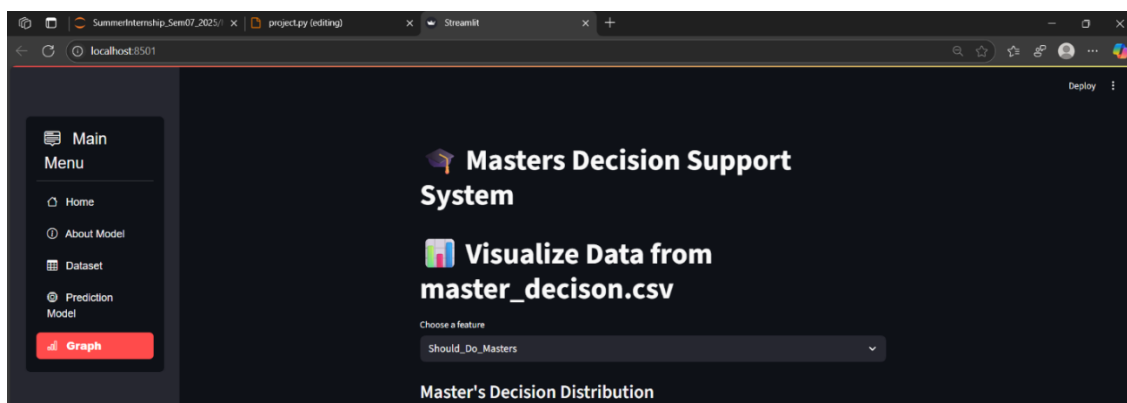
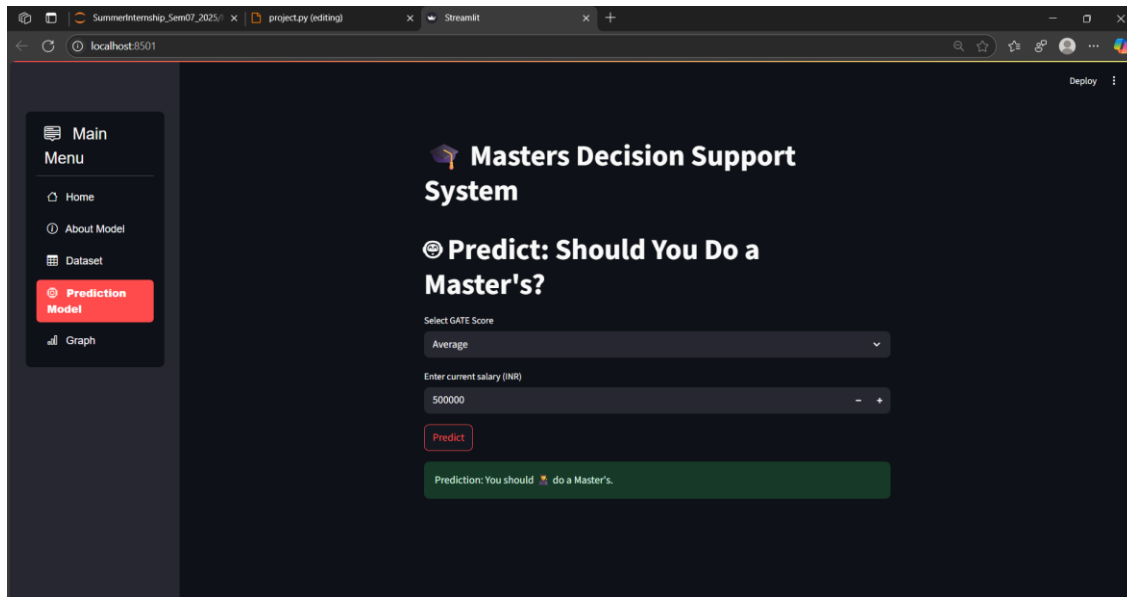
96
97 elif feature == "Salary":
98     st.subheader("Salary Histogram")
99     plt.figure()
100     plt.hist(model_df[feature], bins=15, color="orange", edgecolor="black")
101     plt.xlabel("Salary")
102     plt.ylabel("Frequency")
103     st.pyplot(plt.gcf())
104
105 elif feature == "Should_Do_Masters":
106     st.subheader("Master's Decision Distribution")
107     counts = model_df[feature].value_counts().sort_index()
108     labels = [master_label_map[i] for i in counts.index]
109     plt.figure()
110     plt.pie(counts, labels=labels, autopct='%1.1f%%', colors=["#ff9999", "#66b3ff"])
111     st.pyplot(plt.gcf())
112

```



The screenshot shows the same web application but with the "Dataset" page selected in the sidebar. The main content area displays "Uploaded Dataset (master_decison.csv)" and a table with the following data:

	Student_ID	Student_Name	Gender	Placed	GATE_Score	Salary	Should_Do_Masters
0	S0001	Kunal	Male	No	Good	796246	Yes
1	S0002	Harsh	Male	No	Good	329709	No
2	S0003	Aman	Female	Yes	Poor	1348939	Yes
3	S0004	Ananya	Male	Yes	Poor	794829	No
4	S0005	Kunal	Male	Yes	Poor	696456	No
5	S0006	Nikhil	Male	No	Good	345589	Yes
6	S0007	Aman	Female	No	Good	1196704	Yes
7	S0008	Aman	Female	No	Poor	475037	Yes
8	S0009	Isha	Male	No	Average	508606	No
9	S0010	Ananya	Female	No	Poor	1349458	No



WEEK 2/ Day 11**16 JULY 2024****Objective:**

- Random Forest Project : Fuel consumption
- Conclusion

Project Overview

- This project aims to build an interactive machine learning-powered Streamlit web application that predicts a vehicle's fuel consumption in city, highway, and combined driving conditions.
- By using real-world vehicle data and user-input features such as engine size, transmission type, fuel type, and emissions, the model helps users estimate fuel efficiency and make informed decisions about driving habits, vehicle selection, and cost planning.

Random Forest Algorithm

- This project uses the **Random Forest Regression** algorithm, a powerful ensemble learning method based on decision trees. A random forest builds **multiple decision trees** during training and outputs the **average of their predictions** in regression tasks, making it highly accurate and resistant to overfitting.
- Its ability to handle feature interactions and outliers makes it ideal for real-world vehicle data.

User Input Fields

1. Vehicle Class (VEHICLE CLASS)

This describes the **size or category of the car**.

- **COMPACT**: Small car like a Swift or Baleno
- **SUV**: Sports Utility Vehicle (Creta, Scorpio, etc.)
- **MINICOMPACT**: Very small car
- **PICKUP TRUCK**: Open-back vehicles for transport
- **STATION WAGON**: Like a long hatchback with cargo space

Why it matters: Bigger vehicles usually consume more fuel.

2. Engine Size (L) (ENGINE SIZE)

The size of the engine measured in **litres**. For example:

- **1.2L** → Small engine (fuel efficient)
- **2.0L or 3.5L** → Bigger engine (more power, more fuel)

Why it matters: Larger engines usually consume more fuel but offer more performance.

3. Cylinders (CYLINDERS)

This tells how many **cylinders** the engine has:

- **4 cylinders** is common for regular cars
- **6 or 8 cylinders** in high-performance cars or SUVs

Why it matters: More cylinders = more fuel consumption, usually.

4. Transmission Type (TRANSMISSION)

This shows how the **gear system** works. The values might look like A6, AM, AV, M5, AS8, etc. Here's what they mean:

Code	Meaning
A	Automatic — changes gears automatically.
M	Manual — driver changes gears (with clutch).
AS	Automatic Select Shift — automatic but lets driver shift manually sometimes.
AV	Continuously Variable Transmission (CVT) — smooth automatic gear change without steps.
AM	Automated Manual — a manual transmission with automation.
Numbers like 6 or 8	Number of gear speeds (e.g., A6 = 6-speed automatic)

Why it matters: Automatic or CVT systems can affect how efficiently fuel is used.

5. Fuel Type (FUEL)

Type of fuel the car runs on. Typical options:

- **X** = Regular Gasoline (Petrol)
- **Z** = Premium Gasoline (higher performance petrol)
- **D** = Diesel
- **E** = Ethanol (like E85)
- **N** = Natural Gas (CNG)

Why it matters: Different fuels burn differently, affecting fuel consumption.

6. Emissions (g/km) (EMISSIONS)

This is the amount of **carbon dioxide (CO₂)** the car emits per kilometre, in grams.

- **Lower = better for environment**
- Typical range: 100–400 g/km

Why it matters: More emissions often mean more fuel used.

Use Case

Scenario	How the output helps
Buying a new/used car	Compare efficiency across vehicles.
Estimating fuel cost	Use these numbers to calculate monthly fuel budget.
Eco-conscious driving	Choose cars or settings that reduce city fuel consumption.
Trip planning	Estimate fuel required for highway travel.

```

1 import pandas as pd
2 import streamlit as st
3 from sklearn.preprocessing import LabelEncoder
4 from sklearn.ensemble import RandomForestRegressor
5 import joblib
6
7 st.set_page_config(page_title="Fuel Consumption Predictor", layout="centered")
8 st.title("🚗 Fuel Consumption Predictor")
9 st.markdown("""
10 This app predicts:
11 - **Fuel Consumption (City)**
12 - **Fuel Consumption (Highway)**
13 - **Combined Fuel Consumption**
14
15 based on vehicle specifications.
16 """)
17
18 # Load data
19 data = pd.read_csv("clean_fuel.csv")
20
21 # Encode categorical features
22 class_encoder = LabelEncoder()
23 transmission_encoder = LabelEncoder()
24 fuel_encoder = LabelEncoder()
25
26 data['VEHICLE CLASS'] = class_encoder.fit_transform(data['VEHICLE CLASS'])
27 data['TRANSMISSION'] = transmission_encoder.fit_transform(data['TRANSMISSION'])
28 data['FUEL'] = fuel_encoder.fit_transform(data['FUEL'])
29
30 # Features and targets
31 features = ["VEHICLE CLASS", "ENGINE SIZE", "CYLINDERS", "TRANSMISSION", "FUEL", "EMISSIONS"]
32 target = ["FUEL CONSUMPTION", "Hwy (L/100 km)", "COMB (L/100 km)"]
33 X = data[features]
34 y = data[target]
35
36 @st.cache_resource
37 def train_model():
38     model = RandomForestRegressor(random_state=42)
39     model.fit(X, y)
40     return model
41
42 model = train_model()
43
44 # Streamlit inputs
45 with st.form("prediction_form"):
46     vehicle_class = st.selectbox("Select Vehicle Class", class_encoder.classes_)
47     engine_size = st.number_input("Engine Size (L)", min_value=0.5, max_value=10.0, step=0.1)
48     cylinders = st.number_input("Cylinders", min_value=2, max_value=16, step=1)
49     transmission = st.selectbox("Select Transmission", transmission_encoder.classes_)
50     fuel = st.selectbox("Select Fuel Type", fuel_encoder.classes_)
51     emissions = st.number_input("Enter Emissions (g/km)", min_value=50, max_value=700)
52     submitted = st.form_submit_button("Predict")
53
54 if submitted:
55     try:
56         input_data = pd.DataFrame({
57             "VEHICLE CLASS": [class_encoder.transform([vehicle_class])[0]],
58             "ENGINE SIZE": [engine_size],
59             "CYLINDERS": [cylinders],
60             "TRANSMISSION": [transmission_encoder.transform([transmission])[0]],
61             "FUEL": [fuel_encoder.transform([fuel])[0]],
62             "EMISSIONS": [emissions]
63         })
64
65         predictions = model.predict(input_data)[0]
66
67         st.success("Predicted Fuel Metrics:")
68         st.write(f"**Fuel Consumption (City)**: {predictions[0]:.2f} L/100km")
69         st.write(f"**Fuel Consumption (Highway)**: {predictions[1]:.2f} L/100km")
70         st.write(f"**Combined Fuel Consumption**": {predictions[2]:.2f} L/100km")
71
72     except ValueError as e:
73         st.error(f"Encoding error: {e}")
74

```


Fuel Consumption Predictor

This app predicts:

- Fuel Consumption (City)
- Fuel Consumption (Highway)
- Combined Fuel Consumption

based on vehicle specifications.

Select Vehicle Class: COMPACT

Engine Size (L): 0.50

Cylinders: 2

Select Transmission: A

Select Fuel Type: D

Enter Emissions (g/km): 50

Predict

Predicted Fuel Metrics:

Fuel Consumption (City): 3.72 L/100km

Fuel Consumption (Highway): 3.58 L/100km

Combined Fuel Consumption: 3.66 L/100km

Conclusion

During this internship, I gained hands-on experience in data analytics, Power BI, and machine learning. I learned to work with real-time data from APIs, visualize insights using Python libraries and Power BI, and implement regression and classification models for practical use-cases such as COVID analysis, education decision support, and fuel consumption prediction. This journey enhanced my technical skills and problem-solving approach using data-driven methods. The internship has provided a strong foundation for future roles in data science and business intelligence.

Project Repository

[@NidhiTank_InfolabzSummerInternship2025-Data-Analytics-ML](#)